CYBERSECURITY PROJECT REPORT

FILE INTEGRITY CHECKER

SHA-256 Based File Monitoring System

Submitted by: Muskan Sain

Project Domain: Cybersecurity

Programming Language: Python

Hashing Algorithm: SHA-256

Academic Year: 2026–2027

1. Abstract

The File Integrity Checker is a Python-based cybersecurity project that helps identify whether files have changed after a trusted baseline has been created. It calculates a SHA-256 hash for each regular file, stores the hashes in a JSON baseline, and compares current hashes with the saved values during verification. It reports unchanged, modified, added and missing files. The project is designed as an educational demonstration of file integrity monitoring.

2. Introduction

File integrity is an important part of cybersecurity. Files may be changed accidentally, by faulty software, or by an unauthorized person or program. A file hash acts like a digital fingerprint of file content. When the content changes, its cryptographic hash will normally change as well. This project uses SHA-256, available through Python's hashlib module, to monitor file content.

3. Problem Statement

Users may not immediately notice when important files have been edited, replaced or deleted. Manually comparing files is time-consuming. The project provides a simple automated way to create a reference record and later detect differences.

4. Objectives

Calculate SHA-256 hashes for files.

Create and save a trusted baseline in JSON format.

Verify files against the saved baseline.

Detect modified, newly added and missing files.

Provide clear terminal output and a reproducible notebook demonstration.

5. Scope and Limitations

The program scans a selected directory recursively, hashes regular files in chunks, skips symbolic links and excludes its own baseline file. It is intended for learning and small-scale integrity checks. It does not encrypt files, prevent changes, identify who changed a file, or guarantee that the original baseline was safe. If an attacker can alter both the files and baseline, the result may be unreliable. The baseline should therefore be protected or stored separately.

6. Tools and Technologies

Python 3.9 or newer

hashlib — SHA-256 hashing

json — baseline storage

pathlib and os — directory and file handling

argparse — command-line interface

Jupyter Notebook — interactive demonstration

python-docx — report generation (used only to prepare this report).

7. Working Methodology

Select a directory to monitor.

Read each regular file in binary chunks and calculate its SHA-256 digest.

Save relative file paths, digests and file sizes in a JSON baseline.

During verification, calculate the current hashes again.

Compare the current file list with the saved list.

Classify files as unchanged, modified, added or missing and display the result.

8. SHA-256 Explanation

SHA-256 is a member of the Secure Hash Algorithm 2 family. It produces a 256-bit digest, commonly displayed as 64 hexadecimal characters. The same input content produces the same digest, while a content change produces a different digest with extremely high probability. Hashing is not encryption: it does not hide or recover the original content.

9. Project Files

file_integrity_checker.py — main command-line application.

File_Integrity_Checker.ipynb — step-by-step interactive demonstration.

requirements.txt — documents the runtime dependency choice; no third-party runtime package is needed.

tests/test_file_integrity_checker.py — automated unit tests.

README.md — setup, commands and testing guide.

10. Execution Instructions

Open a terminal in the project folder and check Python with: python --version. Create a sample folder named sample_files and add a text file. Run: python file_integrity_checker.py baseline sample_files. Then run: python file_integrity_checker.py verify sample_files. Edit the text file and verify again; it should be reported as modified. Add another file to test added-file detection, and delete a baseline file to test missing-file detection.

11. Testing Plan and Expected Results

Test case	Action	Expected result
Clean verification	Create baseline; do not change files	PASS; files unchanged
Modification	Edit an existing file	File appears under Modified
Addition	Create a new file	File appears under Added
Deletion	Delete a recorded file	File appears under Missing
Invalid baseline	Use malformed or unsupported JSON	Clear validation error

12. Automated Testing

Run the automated tests from the project root using: python -m unittest discover -s tests -v. The tests use temporary directories and check clean verification, modification, addition, deletion and rejection of an invalid baseline. Record the actual terminal output after running the tests; do not claim a test passed unless it has been executed.

13. How to Interpret Results

PASS — no additions, missing files, modifications or read errors were detected.

WARNING — one or more changes or read errors were detected.

Exit code 0 indicates a clean verification; 1 indicates detected changes/read errors; 2 indicates an input, baseline or execution error.

14. Future Enhancements

Add an optional graphical user interface.

Generate timestamped verification reports.

Support scheduled monitoring.

Add secure baseline signing or external baseline storage.

Add configurable exclusions and file-type filters.

15. Conclusion

The File Integrity Checker demonstrates how cryptographic hashing can support file integrity monitoring. By saving SHA-256 hashes in a baseline and comparing them later, the program can detect modifications, additions and deletions. The project connects Python programming with a practical cybersecurity concept and provides a foundation for more advanced monitoring tools.

16. References

Python Documentation — hashlib: https://docs.python.org/3/library/hashlib.html

Python Documentation — json: https://docs.python.org/3/library/json.html

NIST FIPS PUB 180-4 — Secure Hash Standard: https://csrc.nist.gov/pubs/fips/180-4/upd1/final